Atividade: Da Modelagem à Implementação em Java

Sistema de Vendas da Cantina Escolar

Aluno: Harumi Carvalho Trivia **Data:** 16/09/2026

1. Tarefa 1 – Casos de Uso

1.1 Atores do sistema

• **Cliente:** consulta o cardápio, escolhe produtos, informa quantidades, fecha o pedido e paga.
• **Atendente:** registra a venda, recebe o pagamento e emite comprovante.
• **Gerente:** cadastra produtos, altera preços, atualiza estoques, remove produtos e consulta vendas.

1.2 Casos de uso principais

• Consultar cardápio
• Escolher produtos
• Informar quantidades
• Criar/fechar pedido
• Calcular total
• Realizar e validar pagamento
• Registrar venda
• Emitir comprovante
• Cadastrar produto
• Alterar preço
• Atualizar estoque
• Remover produto
• Consultar vendas

1.3 Caso de uso: Realizar venda

Objetivo: realizar uma venda completa.
Atores: Cliente e Atendente.
Fluxo: 1) consultar cardápio; 2) escolher produtos e quantidades; 3) registrar itens; 4) verificar estoque; 5) calcular total; 6) informar pagamento; 7) validar pagamento; 8) registrar data/hora e atualizar estoque; 9) emitir comprovante.
Exceções: estoque insuficiente impede a finalização; pagamento insuficiente exige novo valor ou forma de pagamento.

1.4 Caso de uso: Gerenciar produtos e estoque

Objetivo: manter cardápio e estoque atualizados.
Ator: Gerente.
Fluxo: cadastrar produto, alterar preço, atualizar estoque ou remover produto do cardápio. O sistema valida e confirma a operação.
Exceções: código duplicado rejeita cadastro; saída maior que o estoque disponível é recusada.

2. Tarefa 2 – Diagrama de Classes

Classes: Produto, ItemPedido, Pedido, Pagamento, TipoPagamento, Venda, Gerente e Atendente.

Diagrama em PlantUML:

```
@startuml
class Produto {
- codigo : int
- nome : String
- categoria : String
- precoUnitario : double
- estoque : int
- ativo : boolean
+ alterarPreco(novoPreco : double)
+ atualizarEstoque(qtd : int)
+ temEstoque(qtd : int) : boolean
+ retirarEstoque(qtd : int)
}
class ItemPedido {
- produto : Produto
- quantidade : int
+ calcularSubtotal() : double
}
class Pedido {
- itens : List<ItemPedido>
- dataHora : LocalDateTime
- finalizado : boolean
+ adicionarItem(produto, quantidade)
+ calcularTotal() : double
+ podeFinalizar() : boolean
+ finalizar(pagamento)
}
class Pagamento {
- tipo : TipoPagamento
- valorPago : double
+ isSuficiente(total) : boolean
+ calcularTroco(total) : double
}
enum TipoPagamento {
DINHEIRO
PIX
CARTAO_CREDITO
CARTAO_DEBITO
}
class Venda {
- pedido : Pedido
- dataHora : LocalDateTime
}
class Gerente
class Atendente

Pedido "1" *-- "1..*" ItemPedido
ItemPedido "*" --> "1" Produto
Pedido "1" --> "1" Pagamento
Pagamento --> TipoPagamento
Venda "1" --> "1" Pedido
Gerente ..> Produto
Atendente ..> Pedido
@enduml
```

Relacionamentos: Pedido–ItemPedido é composição 1 para 1..*; ItemPedido–Produto é associação muitos para um; Pedido–Pagamento é associação 1 para 1 na finalização; Venda–Pedido é associação 1 para 1; Gerente e Atendente dependem das operações correspondentes.

3. Tarefa 3 – Implementação em Java

Produto.java

```
public class Produto {
    private int codigo;
    private String nome, categoria;
    private double precoUnitario;
    private int estoque;
    private boolean ativo;

    public Produto(int codigo, String nome, String categoria,
                   double precoUnitario, int estoque) {
        if (precoUnitario < 0 || estoque < 0)
            throw new IllegalArgumentException("Dados inválidos.");
        this.codigo = codigo; this.nome = nome; this.categoria = categoria;
        this.precoUnitario = precoUnitario; this.estoque = estoque;
        this.ativo = true;
    }
    public int getCodigo(){ return codigo; }
    public String getNome(){ return nome; }
    public String getCategoria(){ return categoria; }
    public double getPrecoUnitario(){ return precoUnitario; }
    public int getEstoque(){ return estoque; }
    public boolean isAtivo(){ return ativo; }
    public void alterarPreco(double p){ if(p<0) throw new IllegalArgumentException(); precoUnitario=p; }
    public void atualizarEstoque(int q){ if(estoque+q<0) throw new IllegalArgumentException("Estoque insuficiente."); estoque+=q
    public boolean temEstoque(int q){ return ativo && q>0 && estoque>=q; }
    public void retirarEstoque(int q){ if(!temEstoque(q)) throw new IllegalArgumentException("Estoque insuficiente."); estoque-=
    public void removerDoCardapio(){ ativo=false; }
}
```

ItemPedido.java

```
public class ItemPedido {
    private Produto produto;
    private int quantidade;

    public ItemPedido(Produto produto, int quantidade) {
        if(produto == null || quantidade <= 0) throw new IllegalArgumentException("Dados inválidos.");
        this.produto = produto; this.quantidade = quantidade;
    }
    public Produto getProduto(){ return produto; }
    public int getQuantidade(){ return quantidade; }
    public void setQuantidade(int q){ if(q<=0) throw new IllegalArgumentException(); quantidade=q; }
    public double calcularSubtotal(){ return produto.getPrecoUnitario() * quantidade; }
}
```

TipoPagamento.java

```
public enum TipoPagamento {
    DINHEIRO, PIX, CARTAO_CREDITO, CARTAO_DEBITO
}
```

Pagamento.java

```
public class Pagamento {
    private TipoPagamento tipo;
    private double valorPago;

    public Pagamento(TipoPagamento tipo, double valorPago) {
        if(tipo == null || valorPago < 0) throw new IllegalArgumentException("Pagamento inválido.");
        this.tipo=tipo; this.valorPago=valorPago;
    }
    public TipoPagamento getTipo(){ return tipo; }
    public double getValorPago(){ return valorPago; }
    public boolean isSuficiente(double total){ return valorPago >= total; }
    public double calcularTroco(double total){ return isSuficiente(total) ? valorPago-total : 0; }
}
```

Pedido.java

```
import java.time.LocalDateTime;
import java.util.*;

public class Pedido {
    private List<ItemPedido> itens = new ArrayList<>();
    private LocalDateTime dataHora = LocalDateTime.now();
    private boolean finalizado = false;

    public void adicionarItem(Produto p, int qtd) {
        if(finalizado) throw new IllegalStateException("Pedido já finalizado.");
        if(!p.temEstoque(qtd)) throw new IllegalArgumentException("Estoque insuficiente.");
        itens.add(new ItemPedido(p,qtd));
    }
    public double calcularTotal() {
        double total=0;
        for(ItemPedido i: itens) total += i.calcularSubtotal();
        return total;
    }
    public boolean podeFinalizar() {
        if(itens.isEmpty()) return false;
        for(ItemPedido i: itens)
            if(!i.getProduto().temEstoque(i.getQuantidade())) return false;
        return true;
    }
    public void finalizar(Pagamento pagamento) {
        if(!podeFinalizar()) throw new IllegalStateException("Pedido inválido.");
        double total=calcularTotal();
        if(!pagamento.isSuficiente(total)) throw new IllegalArgumentException("Pagamento insuficiente.");
        for(ItemPedido i: itens) i.getProduto().retirarEstoque(i.getQuantidade());
        finalizado=true;
    }
    public List<ItemPedido> getItens(){ return Collections.unmodifiableList(itens); }
    public LocalDateTime getDataHora(){ return dataHora; }
}
```

Main.java

```
public class Main {
    public static void main(String[] args) {
        Produto coxinha = new Produto(1,"Coxinha","Lanche",6.50,20);
        Produto suco = new Produto(2,"Suco","Bebida",5.00,15);
        Produto brigadeiro = new Produto(3,"Brigadeiro","Doce",3.50,10);

        Pedido pedido = new Pedido();
        pedido.adicionarItem(coxinha,2);
        pedido.adicionarItem(suco,1);

        double total = pedido.calcularTotal();
        Pagamento pagamento = new Pagamento(TipoPagamento.DINHEIRO,20.00);
        pedido.finalizar(pagamento);

        System.out.println("===== COMPROVANTE =====");
        for(ItemPedido i: pedido.getItens())
            System.out.printf("%d x %s = R$ %.2f%n",
                i.getQuantidade(), i.getProduto().getNome(), i.calcularSubtotal());
        System.out.printf("TOTAL: R$ %.2f%n",total);
        System.out.println("Pagamento: "+pagamento.getTipo());
        System.out.printf("Valor pago: R$ %.2f%n",pagamento.getValorPago());
        System.out.printf("Troco: R$ %.2f%n",pagamento.calcularTroco(total));
        System.out.println("Venda finalizada com sucesso.");
    }
}
```

4. Exemplo de saída

```
===== COMPROVANTE =====
2 x Coxinha = R$ 13,00
1 x Suco = R$ 5,00
TOTAL: R$ 18,00
Pagamento: DINHEIRO
Valor pago: R$ 20,00
Troco: R$ 2,00
Venda finalizada com sucesso.
```

5. Conclusão

A implementação atende às regras principais: controle de produtos e estoque, itens e subtotais, cálculo do total, validação do pagamento, troco, data e hora do pedido e demonstração de uma venda completa.